

Finyl-DCP — AWS “Option 2” Deployment Runbook (ECS Fargate)

Option 2 = Amazon ECS on Fargate (backend) + Application Load Balancer + Amazon RDS for PostgreSQL (database) + Netlify (frontend).

Use this option when **AWS App Runner is not available in your target region.**

App Runner is **not offered in af-south-1 (Cape Town)** — the nearest AWS region to Kenya — so this runbook deploys the same backend container on **ECS Fargate** behind an **Application Load Balancer (ALB)** instead. Everything is driven by environment/config; **no application code changes are required.**

This runbook is precise and copy-pasteable, ordered top-to-bottom. The current live Abacus deployment (<https://finyl-dcp.abacusai.cloud>) keeps working unchanged; you can run both in parallel and cut over only after the verification checklist in section (p) passes.

Placeholders. Everything in `<ANGLE_BRACKETS>` is a value you supply. Common ones: `<ACCOUNT_ID>` (your 12-digit AWS account id), `<REGION>` (default `af-south-1` , Cape Town — see residency note in (a)), `<RDS_ENDPOINT>` , `<IMAGE_URI>` (the ECR image URI from step f), `<SUFFIX>` (the random suffix AWS appends to Secrets Manager ARNs), `<ALB_DNS>` (the ALB’s DNS name).

Editable JSON templates. This option ships two ready-to-edit ECS task definitions: `deploy/aws/ecs-task-web.json` and `deploy/aws/ecs-task-scheduler.json` . Unlike the App Runner input, they contain **no comment keys** — they are accepted by `aws ecs register-task-definition --cli-input-json file://...` as soon as you fill the placeholders. All explanation lives in this document.

(q) TL;DR — hand this to an operator or AI agent

Run these in order. Each step is expanded below with full commands, placeholders, and rationale. **Do not skip (a) prerequisites or (p) verification.**

- a. Prereqs: AWS account + CLI + Docker; pick REGION=af-south-1; Netlify account. Carry over the crypto keys (JWT_SECRET, FIELD_ENCRYPTION_KEY, PII_ENCRYPTION_KEY) from the current Abacus backend/.env UNCHANGED if you will migrate existing data.
- b. Networking: pick a VPC + 2 subnets; create 3 security groups – ALB SG (443/80 from internet), ECS tasks SG (8000 from ALB SG only), RDS SG (5432 from ECS SG).
- c. Create RDS Postgres (encrypted) reachable from the ECS tasks SG.
- d. Store platform secrets in Secrets Manager (DATABASE_URL + crypto keys + Daraja defaults + integration keys).
- e. Create the ECS task EXECUTION role (ECR pull + logs + read finyl-dcp/* secrets) and the TASK role (least privilege; S3 put for backups).
- f. Build the backend image (linux/amd64) and push to ECR.
- g. Create the ECS Fargate cluster + the CloudWatch log group.
- h. Register TWO task definitions and create TWO services from the SAME image:
 - finyl-web – SCHEDULER_ENABLED=false, behind the ALB, desiredCount 1-2 (scalable)
 - finyl-scheduler – SCHEDULER_ENABLED=true, NOT on the ALB, desiredCount EXACTLY 1
 (Two services because the scheduler is in-process; see the box in (h).)
- i. Create the ALB + target group (health check /api/health) + HTTPS listener (ACM cert) with HTTP->HTTPS redirect; attach finyl-web to the target group.
- j. Bootstrap the DB with a one-off `aws ecs run-task` running scripts/run_migrations.py.
- k. (Optional) Migrate data from Abacus with pg_dump/pg_restore – copy the crypto keys UNCHANGED or PII won't decrypt.
- l. Point the Netlify frontend at the ALB via the /api/* proxy in netlify.toml.
- m. Repoint M-Pesa/Daraja callbacks at the ALB URL; flip go-live env vars.
- n. (Optional) Autoscale finyl-web on CPU. NEVER scale finyl-scheduler beyond 1.
- o. Cost note: ECS tasks + always-on ALB + RDS are billed by AWS.
- p. Run the PRE-CUTOVER VERIFICATION CHECKLIST. Parallel-run, then decommission Abacus.

(a) Prerequisites

1. **AWS account** with permissions for ECS, EC2 (VPC/ELB), ECR, RDS, Secrets Manager, IAM, CloudWatch Logs, ACM, and S3.
2. **AWS CLI v2** installed and configured (`aws configure`). All commands below pass `--region <REGION>` explicitly so you can override.
3. **Docker** installed locally (to build and push the backend image).
4. **jq** (used to read a few CLI outputs) — optional but convenient.
5. **Netlify account** (free tier is fine for the pilot) connected to the GitHub repo.
6. **Region — data residency note (owner's decision)**. AWS has **no Kenya region**. The nearest is **af-south-1 (Cape Town, South Africa)**, used as the default throughout this runbook and the reason for choosing ECS (App Runner is unavailable there). Kenya's Data Protection Act (ODPC) permits cross-border transfer under conditions; **whether to store Kenyan borrower PII in af-south-1 (or eu-west-1, etc.) is a business/legal decision for the data controller — it is not a technical blocker**. Set `<REGION>` once and reuse it everywhere.
7. **Carry-over crypto keys (critical if migrating data)**. If you will migrate existing borrower data from Abacus (step k), you **MUST** reuse the SAME `FIELD_ENCRYPTION_KEY`, `PII_ENCRYPTION_KEY`, and `JWT_SECRET` values from the current Abacus `backend/.env`. Different keys ⇒ encrypted PII (national IDs) cannot be decrypted and all existing sessions are invalidated. Retrieve them from

the Abacus deployment before you begin and store them in Secrets Manager in step (d). For a **fresh pilot with no data migration**, generate new keys instead.

```
# Set these once in your shell for the rest of the runbook:
export REGION=af-south-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export CLUSTER=finyl-dcp
echo "Account=$ACCOUNT_ID Region=$REGION Cluster=$CLUSTER"
```

(b) Networking — VPC, subnets, and security groups

ECS Fargate tasks run inside a VPC with elastic network interfaces. You need a VPC, **two subnets in different AZs** (the ALB requires ≥ 2 AZs), and **three security groups** that form a strict chain:

```
internet —443/80→ [ALB SG] —8000→ [ECS tasks SG] —5432→ [RDS SG]
```

Only the ALB is exposed to the internet; the tasks accept traffic **only** from the ALB SG; RDS accepts traffic **only** from the ECS tasks SG.

b.1 — Choose the VPC and subnets

```
# Use your default VPC (or substitute your own VPC id).
VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true \
  --query "Vpcs[0].VpcId" --output text --region "$REGION")

# Pick two subnets in different AZs. This grabs the first two; verify they are in
# distinct AZs (the ALB and Fargate both need multi-AZ).
SUBNET_IDS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID \
  --query "Subnets[].SubnetId" --output text --region "$REGION")
SUBNET_A=$(echo $SUBNET_IDS | awk '{print $1}')
SUBNET_B=$(echo $SUBNET_IDS | awk '{print $2}')
echo "VPC=$VPC_ID SubnetA=$SUBNET_A SubnetB=$SUBNET_B"
```

b.2 — Create the three security groups

```
# 1) ALB SG – public HTTPS (443) and HTTP (80, for the redirect) from the internet.
ALB_SG_ID=$(aws ec2 create-security-group \
  --group-name finyl-dcp-alb-sg \
  --description "Finyl-DCP ALB – 443/80 from internet" \
  --vpc-id "$VPC_ID" --query GroupId --output text --region "$REGION")
aws ec2 authorize-security-group-ingress --group-id "$ALB_SG_ID" \
  --protocol tcp --port 443 --cidr 0.0.0.0/0 --region "$REGION"
aws ec2 authorize-security-group-ingress --group-id "$ALB_SG_ID" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0 --region "$REGION"

# 2) ECS tasks SG – app port 8000 ONLY from the ALB SG.
ECS_SG_ID=$(aws ec2 create-security-group \
  --group-name finyl-dcp-ecs-sg \
  --description "Finyl-DCP ECS tasks – 8000 from ALB SG only" \
  --vpc-id "$VPC_ID" --query GroupId --output text --region "$REGION")
aws ec2 authorize-security-group-ingress --group-id "$ECS_SG_ID" \
  --protocol tcp --port 8000 --source-group "$ALB_SG_ID" --region "$REGION"

# 3) RDS SG – Postgres 5432 ONLY from the ECS tasks SG.
RDS_SG_ID=$(aws ec2 create-security-group \
  --group-name finyl-dcp-rds-sg \
  --description "Finyl-DCP RDS – 5432 from ECS tasks SG only" \
  --vpc-id "$VPC_ID" --query GroupId --output text --region "$REGION")
aws ec2 authorize-security-group-ingress --group-id "$RDS_SG_ID" \
  --protocol tcp --port 5432 --source-group "$ECS_SG_ID" --region "$REGION"

echo "ALB_SG=$ALB_SG_ID ECS_SG=$ECS_SG_ID RDS_SG=$RDS_SG_ID"
```

b.3 — Where do the tasks run? Two networking models

Fargate tasks need a route to pull the image from ECR and read Secrets Manager.

Choose **one** model and use it consistently for the services in (h) and the run-task in (j):

- **Public subnets + `assignPublicIp=ENABLED` (recommended for a pilot — simpler, cheaper).** Tasks get a public IP and reach ECR/Secrets Manager directly over the internet gateway. **No NAT gateway needed.** The tasks are still **not reachable inbound** from the internet because the ECS tasks SG only allows 8000 from the ALB SG. This is the simplest path and is what the `create-service` commands below use. *Tradeoff:* tasks have public IPs (egress only); acceptable for a short pilot.
- **Private subnets + NAT gateway (recommended for production).** Tasks have no public IP; outbound to ECR/Secrets Manager goes through a **NAT gateway** in a public subnet (extra hourly + data cost), or through **VPC endpoints** for ECR, S3, CloudWatch Logs, and Secrets Manager (no NAT, more setup). Stronger isolation. To use this model: put the tasks in private subnets and set `assignPublicIp=DISABLED` in the `awsVpcConfiguration` of the `create-service` (and `run-task`) commands, and ensure NAT/VPC-endpoint egress exists.

The commands below default to the **public-subnet pilot** model. The one line to change for the private model is called out inline.

(c) Create the RDS PostgreSQL instance

Finyl-DCP runs on PostgreSQL 14+. We provision an **encrypted** instance reachable only from the ECS tasks SG created in (b.2). (These steps mirror Option 1 (b).)

c.1 — Generate and store the DB password

```
DB_PASSWORD=$(python3 -c "import secrets;print(secrets.token_urlsafe(24))")
aws secretsmanager create-secret \
  --name finyl-dcp/DB_PASSWORD \
  --description "Finyl-DCP RDS master password" \
  --secret-string "$DB_PASSWORD" \
  --region "$REGION"
```

c.2 — Create the DB subnet group and the instance

```
aws rds create-db-subnet-group \
  --db-subnet-group-name finyl-dcp-subnets \
  --db-subnet-group-description "Finyl-DCP RDS subnets" \
  --subnet-ids $SUBNET_IDS --region "$REGION"

aws rds create-db-instance \
  --db-instance-identifier finyl-dcp-db \
  --db-name finyl_dcp \
  --engine postgres \
  --engine-version 16 \
  --db-instance-class db.t4g.micro \
  --allocated-storage 20 \
  --storage-type gp3 \
  --storage-encrypted \
  --master-username finyl \
  --master-user-password "$DB_PASSWORD" \
  --no-publicly-accessible \
  --vpc-security-group-ids "$RDS_SG_ID" \
  --db-subnet-group-name finyl-dcp-subnets \
  --backup-retention-period 7 \
  --region "$REGION"

# storage-encrypted uses the default aws/rds KMS key. To use a customer-managed
# key add: --kms-key-id <KMS_KEY_ARN>

aws rds wait db-instance-available --db-instance-identifier finyl-dcp-db --region "$REGION"
RDS_ENDPOINT=$(aws rds describe-db-instances --db-instance-identifier finyl-dcp-db \
  --query "DBInstances[0].Endpoint.Address" --output text --region "$REGION")
echo "RDS endpoint: $RDS_ENDPOINT"
```

- `db.t4g.micro` + 20 GB gp3 is a reasonable **pilot** size. (Cost estimate: roughly ~US\$12-15/mo for the instance + storage; verify current pricing. Values are estimates.)
- **PostgreSQL 15 or 16** are both fine (the app targets 14+).
- The instance is private (`--no-publicly-accessible`); the ECS tasks reach it inside the VPC. **TLS is enforced via the URL query string — `?sslmode=require` — with no code change** (psycopg2 honours it). Build the connection URL (used in d and j):

```
postgresql://finyl:<DB_PASSWORD>@<RDS_ENDPOINT>:5432/finyl_dcp?sslmode=require
```

(d) Store platform secrets in Secrets Manager

Only platform-level secrets go here. Per-tenant M-Pesa/Daraja credentials are stored **per-tenant, encrypted in the database** (`TenantIntegrationConfig`) — they are **NOT** placed in Secrets Manager. The Daraja values below are the *platform default* fallbacks only.

```
# Assemble the DATABASE_URL from the RDS endpoint + the DB password secret.
DB_PASSWORD=$(aws secretsmanager get-secret-value --secret-id finyl-dcp/DB_PASSWORD \
  --query SecretString --output text --region "$REGION")
DATABASE_URL="postgresql://finyl:${DB_PASSWORD}@${RDS_ENDPOINT}:5432/finyl_dcp?sslmode=require"

create_secret () { # name value
  aws secretsmanager create-secret --name "$1" --secret-string "$2" --region
"$REGION" >/dev/null \
  && echo "  created $1"
}

# --- Core platform secrets ---
create_secret finyl-dcp/DATABASE_URL "$DATABASE_URL"
```

CRITICAL — crypto keys carry-over. These three keys decide whether existing encrypted data is readable:

- **Migrating data from Abacus (step k):** copy the EXACT current values of `JWT_SECRET`, `FIELD_ENCRYPTION_KEY`, and `PII_ENCRYPTION_KEY` from the Abacus `backend/.env` into the secrets below **unchanged**. Different keys ⇒ encrypted PII (e.g. `borrowers.national_id`) will NOT decrypt and all sessions invalidate.
- **Fresh pilot with no data migration:** generate new keys as shown below.

```
# CRYPTO KEYS – paste EXACT Abacus values if migrating (step k); else generate fresh:
create_secret finyl-dcp/JWT_SECRET "$(python3 -c 'import
secrets;print(secrets.token_urlsafe(48))')"
```

```
create_secret finyl-dcp/FIELD_ENCRYPTION_KEY "$(python3 -c 'from cryptography.fernet
import Fernet;print(Fernet.generate_key().decode())')"
```

```
create_secret finyl-dcp/PII_ENCRYPTION_KEY "$(python3 -c 'from cryptography.fernet
import Fernet;print(Fernet.generate_key().decode())')"
```

```
# M-Pesa callback path token (unguessable segment in webhook URLs; not a crypto
secret)
create_secret finyl-dcp/MPESA_CALLBACK_TOKEN "$(python3 -c 'import
secrets;print(secrets.token_hex(12))')"
```

```
# --- Daraja PLATFORM defaults (per-tenant creds live encrypted in the DB) ---
create_secret finyl-dcp/DARAJA_CONSUMER_KEY "<PLATFORM_DARAJA_CONSUMER_KEY>"
create_secret finyl-dcp/DARAJA_CONSUMER_SECRET "<PLAT-
FORM_DARAJA_CONSUMER_SECRET>"
create_secret finyl-dcp/DARAJA_SHORTCODE "<PLATFORM_DARAJA_SHORTCODE>"
create_secret finyl-dcp/DARAJA_PASSKEY "<PLATFORM_DARAJA_PASSKEY>"
create_secret finyl-dcp/DARAJA_SECURITY_CREDENTIAL "<PLAT-
FORM_DARAJA_SECURITY_CREDENTIAL>"
```

```
# --- Other integration secrets (fill real values, or a harmless placeholder) ---
create_secret finyl-dcp/LLM_API_KEY "<LLM_API_KEY>"
create_secret finyl-dcp/UWAZII_USERNAME "<UWAZII_USERNAME>"
create_secret finyl-dcp/UWAZII_PASSWORD "<UWAZII_PASSWORD>"
create_secret finyl-dcp/UWAZII_SENDER_ID "<UWAZII_SENDER_ID>"
create_secret finyl-dcp/SMS_API_KEY "<SMS_API_KEY>"
create_secret finyl-dcp/CRB_API_KEY "<CRB_API_KEY>"
create_secret finyl-dcp/CRB_USERNAME "<CRB_USERNAME>"
create_secret finyl-dcp/CRB_PASSWORD "<CRB_PASSWORD>"
create_secret finyl-dcp/EKYC_USERNAME "<EKYC_USERNAME>"
create_secret finyl-dcp/EKYC_PASSWORD "<EKYC_PASSWORD>"
create_secret finyl-dcp/EKYC_STRATEGY_ID "<EKYC_STRATEGY_ID>"
```

Get every secret's full ARN (with the `<SUFFIX>`) — you paste these into the `valueFrom` fields of the two task-definition JSON files:

```
aws secretsmanager list-secrets --region "$REGION" \
--query "SecretList[?starts_with(Name,'finyl-dcp/')].[Name,ARN]" --output table
```

The task JSON templates use ARNs shaped

```
arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:finyl-dcp/<NAME>-<SUFFIX> .
```

Replace `<SUFFIX>` with the real 6-character suffix from the ARNs above (each secret has its own suffix).

(e) IAM roles — task execution role and task role

ECS Fargate uses **two** roles:

- **Execution role** (`finyl-dcp-ecs-execution`) — used by the ECS agent to pull the image from ECR, write CloudWatch Logs, and **fetch the secrets** referenced in the task definition's `secrets` array. Assumed by `ecs-tasks.amazonaws.com` .

- **Task role** (`finyl-dcp-ecs-task`) — assumed by your application code at runtime. Least privilege; here it only needs `s3:PutObject` to the backups bucket (used by `deploy/backup_db.sh` if you run backups as a task).

e.1 — Trust policy (shared by both roles)

```
cat > /tmp/ecs-tasks-trust.json <<'JSON'
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Principal": { "Service": "ecs-tasks.amazonaws.com" },
    "Action": "sts:AssumeRole" } ] }
JSON
```

e.2 — Execution role (ECR pull + logs + read finyl-dcp/* secrets)

```
aws iam create-role --role-name finyl-dcp-ecs-execution \
  --assume-role-policy-document file:///tmp/ecs-tasks-trust.json

# AWS-managed base policy: ECR pull + CloudWatch Logs.
aws iam attach-role-policy --role-name finyl-dcp-ecs-execution \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy

# Least-privilege addition: read ONLY finyl-dcp/* secrets (needed for the `secrets`
# array in the task definitions).
cat > /tmp/ecs-exec-secrets.json <<'JSON'
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": "secretsmanager:GetSecretValue",
    "Resource": "arn:aws:secretsmanager:${REGION}:${ACCOUNT_ID}:secret:finyl-dcp/*" }
  ] }
JSON
aws iam put-role-policy --role-name finyl-dcp-ecs-execution \
  --policy-name finyl-dcp-secrets-read \
  --policy-document file:///tmp/ecs-exec-secrets.json
```

e.3 — Task role (least privilege — S3 put for backups)

```
aws iam create-role --role-name finyl-dcp-ecs-task \
  --assume-role-policy-document file:///tmp/ecs-tasks-trust.json

# Only grant what the app actually needs. For DB backups to S3 (section k):
cat > /tmp/ecs-task-policy.json <<'JSON'
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": ["s3:PutObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::finyl-dcp-backups-${ACCOUNT_ID}",
      "arn:aws:s3:::finyl-dcp-backups-${ACCOUNT_ID}/*" ] }
  ] }
JSON
aws iam put-role-policy --role-name finyl-dcp-ecs-task \
  --policy-name finyl-dcp-backups-s3 \
  --policy-document file:///tmp/ecs-task-policy.json
```

Both task JSON files reference these roles as

```
arn:aws:iam::<ACCOUNT_ID>:role/finyl-dcp-ecs-execution (executionRoleArn) and
arn:aws:iam::<ACCOUNT_ID>:role/finyl-dcp-ecs-task (taskRoleArn). Only the
<ACCOUNT_ID> needs substituting.
```

(f) Build the backend image and push to ECR

```
aws ecr create-repository --repository-name finyl-dcp-backend --region "$REGION"

ECR_URI="${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/finyl-dcp-backend"

aws ecr get-login-password --region "$REGION" \
  | docker login --username AWS --password-stdin "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com"

# Build from the backend/ directory (Dockerfile lives there). Fargate runs
# linux/amd64 – build for that platform explicitly if you are on Apple Silicon.
cd backend
docker build --platform linux/amd64 -t finyl-dcp-backend:latest .
docker tag finyl-dcp-backend:latest "${ECR_URI}:latest"
docker push "${ECR_URI}:latest"
cd ..

echo "Image URI to paste into both task JSON files as <IMAGE_URI>:"
echo "${ECR_URI}:latest"
```

The backend Dockerfile listens on `$PORT` (default 8000), matching the container port in the task definitions. It also bundles `scripts/` and `migrations/` for the one-off migration task in step (j).

(g) Create the ECS Fargate cluster and the log group

```
aws ecs create-cluster --cluster-name "$CLUSTER" \
  --capacity-providers FARGATE FARGATE_SPOT \
  --region "$REGION"

# CloudWatch log group referenced by both task definitions (awslogs-group).
aws logs create-log-group --log-group-name /ecs/finyl-dcp --region "$REGION"
# Optional retention (e.g. 30 days):
aws logs put-retention-policy --log-group-name /ecs/finyl-dcp \
  --retention-in-days 30 --region "$REGION"
```

(h) Register two task definitions and create two services

Why TWO services from the SAME image

*Finyl-DCP runs its background jobs with an **in-process APScheduler**. On startup (`app/main.py`) the app calls `start_scheduler()` , which — **only when** `SCHEDULER_ENABLED` **is true** — registers three recurring jobs: auto-reconcile, webhook-retry, and webhook-purge. That scheduler lives **inside each web process**.*

*If you run the web service behind the ALB with `SCHEDULER_ENABLED=true` and scale it to *N* tasks, you get **N copies of every scheduled job running in parallel** — double reconciliation, duplicate webhook retries, etc. (APScheduler's `max_instances=1` only prevents overlap **within a single process**, not across tasks.)*

*The fix requires **no code change**: split into two services from the same image —*

- **finyl-web** — `SCHEDULER_ENABLED=false` , behind the ALB, freely scalable (desiredCount 1-2+). Serves the API; runs **no** background jobs.
- **finyl-scheduler** — `SCHEDULER_ENABLED=true` , **not** on the ALB, **desiredCount EXACTLY 1**. Runs the background jobs and nothing else faces it.

Both use the same container image, same DATABASE_URL, and same secrets. See (n) for the autoscaling caveat.

h.1 — Fill in and register the task definitions

Edit `deploy/aws/ecs-task-web.json` and `deploy/aws/ecs-task-scheduler.json` , replacing:

- `<ACCOUNT_ID>` — in the two role ARNs.
- `<IMAGE_URI>` — the `${ECR_URI}:latest` value from step (f).
- `<REGION>` — in the secret ARNs and `awslogs-region` (default `af-south-1`).
- `<SUFFIX>` — the real suffix on each Secrets Manager ARN from step (d).
- The plain `environment` values you want to change now vs. at go-live: `ALLOWED_ORIGINS` , `DARAJA_CALLBACK_BASE_URL` , `DARAJA_ENVIRONMENT` (start `sandbox`), `SAFARICOM_IP_ENFORCE` (start `log`), and the `LLM_*` values.

The only differences between the two files are: `family/container name` (`finyl-web` vs `finyl-scheduler`), `SCHEDULER_ENABLED` (`false` vs `true`), size (web 512 / 1024 , scheduler 256 / 512), the log `stream-prefix` , and that the scheduler file has **no portMappings** (it takes no inbound traffic).

```
aws ecs register-task-definition \
  --cli-input-json file://deploy/aws/ecs-task-web.json --region "$REGION"

aws ecs register-task-definition \
  --cli-input-json file://deploy/aws/ecs-task-scheduler.json --region "$REGION"
```

h.2 — Create the finyl-web service (behind the ALB)

Create this **after** the target group exists (section i) — or create the ALB/target group first, then run this. It references `$TG_ARN` from (i).

```
aws ecs create-service \
  --cluster "$CLUSTER" \
  --service-name finyl-web \
  --task-definition finyl-web \
  --launch-type FARGATE \
  --desired-count 1 \
  --network-configuration "awsvpcConfiguration={subnets=[$SUBNET_A,$SUBNET_B],securityGroups=[$ECS_SG_ID],assignPublicIp=ENABLED}" \
  --load-balancers "targetGroupArn=$TG_ARN,containerName=finyl-web,containerPort=8000" \
  --health-check-grace-period-seconds 60 \
  --region "$REGION"
# Private-subnet model: set assignPublicIp=DISABLED (and ensure NAT/VPC endpoints).
```

h.3 — Create the finyl-scheduler service (NOT on the ALB, exactly 1 task)

```
aws ecs create-service \
  --cluster "$CLUSTER" \
  --service-name finyl-scheduler \
  --task-definition finyl-scheduler \
  --launch-type FARGATE \
  --desired-count 1 \
  --network-configuration "awsvpcConfiguration={subnets=[$SUBNET_A,$SUBNET_B],securityGroups=[$ECS_SG_ID],assignPublicIp=ENABLED}" \
  --region "$REGION"
# NO --load-balancers here. NEVER set --desired-count above 1 for this service (n).
```

Ephemeral storage caveat (`STORAGE_DIR=/app/storage`). Fargate task storage is **per-task and ephemeral** — it is lost when a task restarts and is **not shared** between web tasks. For a single-web-task pilot this is acceptable, but if you scale `finyl-web` beyond 1 task, document uploads written to the local disk won't be visible across tasks. For multi-task or durable document storage, back `STORAGE_DIR` with **S3** (or mount **EFS** on the tasks). This is a deployment/config decision — no application code change is implied here.

(i) Application Load Balancer + target group + HTTPS listener

The ALB terminates TLS and forwards to the `finyl-web` tasks on port 8000. It needs an **ACM certificate** for your API domain.

i.1 — Request/obtain an ACM certificate

```
# Request a public cert for your API hostname (DNS-validated). Create the shown
# CNAME with your registrar, then wait for status ISSUED.
CERT_ARN=$(aws acm request-certificate \
  --domain-name api.yourbank.co.ke \
  --validation-method DNS \
  --query CertificateArn --output text --region "$REGION")
echo "Add the DNS validation CNAME shown here, then wait for ISSUED:"
aws acm describe-certificate --certificate-arn "$CERT_ARN" \
  --query "Certificate.DomainValidationOptions" --region "$REGION"
```

No custom domain yet? You can bring the stack up HTTP-only first (create just the HTTP listener in i.3 forwarding to the target group) and add the HTTPS listener once the cert is ISSUED. Production/M-Pesa callbacks require HTTPS.

i.2 — Create the ALB and the target group

```
ALB_ARN=$(aws elbv2 create-load-balancer \
  --name finyl-dcp-alb \
  --subnets $SUBNET_A $SUBNET_B \
  --security-groups "$ALB_SG_ID" \
  --scheme internet-facing --type application \
  --query "LoadBalancers[0].LoadBalancerArn" --output text --region "$REGION")

ALB_DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns "$ALB_ARN" \
  --query "LoadBalancers[0].DNSName" --output text --region "$REGION")
echo "ALB DNS: $ALB_DNS"

# Target group: type=ip (required for Fargate awsvpc), port 8000, health /api/health.
TG_ARN=$(aws elbv2 create-target-group \
  --name finyl-dcp-web-tg \
  --protocol HTTP --port 8000 --target-type ip \
  --vpc-id "$VPC_ID" \
  --health-check-protocol HTTP --health-check-path /api/health \
  --health-check-interval-seconds 30 --healthy-threshold-count 2 \
  --matcher HttpCode=200 \
  --query "TargetGroups[0].TargetGroupArn" --output text --region "$REGION")
echo "Target group ARN: $TG_ARN"
```

i.3 — Listeners: HTTPS (forward) + HTTP (redirect to HTTPS)

```
# HTTPS:443 -> forward to the target group.
aws elbv2 create-listener \
  --load-balancer-arn "$ALB_ARN" \
  --protocol HTTPS --port 443 \
  --certificates CertificateArn="$CERT_ARN" \
  --ssl-policy ELBSecurityPolicy-TLS13-1-2-2021-06 \
  --default-actions Type=forward,TargetGroupArn="$TG_ARN" \
  --region "$REGION"

# HTTP:80 -> permanent redirect to HTTPS.
aws elbv2 create-listener \
  --load-balancer-arn "$ALB_ARN" \
  --protocol HTTP --port 80 \
  --default-actions '[{"Type":"redirect","RedirectConfig":{"Protocol":"HTTPS","Port":"443","StatusCode":"HTTP_301"}}]' \
  --region "$REGION"
```

Now create the finyl-web service (h.2) if you haven't — it registers its tasks into `$TG_ARN` automatically. The backend URL is `https://api.yourbank.co.ke` (point that DNS record at `$ALB_DNS` via a CNAME/ALIAS), or `https://$ALB_DNS` for a quick test before DNS. Health check target: `GET /api/health → {"status":"ok",...}`.

(j) Bootstrap the database (fresh pilot)

Skip this section if you are migrating existing data — do section (k) instead.

The numbered migrations (`backend/migrations/002...016`) are **additive** changes on top of the base tables defined by the SQLAlchemy models. Bootstrapping a brand-new DB is: (1) create base tables from the models, (2) run the migration runner.

Primary approach — a one-off `aws ecs run-task` **with a command override**. This runs the migration inside the VPC using the same image, roles, and secrets, so it reaches the private RDS with no bastion. It uses the **finyl-web** task definition but overrides the container command.

```
# 1) Create base tables (fresh DB only), then 2) apply additive migrations – in one
# container invocation via a bash -lc override.
aws ecs run-task \
  --cluster "$CLUSTER" \
  --launch-type FARGATE \
  --task-definition finyl-web \
  --count 1 \
  --network-configuration "awsvpcConfiguration={subnets=[$SUBNET_A,$SUBNET_B],securityGroups=[$ECS_SG_ID],assignPublicIp=ENABLED}" \
  --overrides '{
  "containerOverrides": [{
    "name": "finyl-web",
    "environment": [{"name":"AUTO_CREATE_TABLES","value":"true"}],
    "command": ["bash","-lc","python -c \"from app.core.database import Base, engine, ensure_schema; from app import models; ensure_schema(); Base.metadata.create_all(bind=engine); print(\\\"\\\"base tables created\\\"\\\")\" && python scripts/run_migrations.py"]
  }]
}' \
  --region "$REGION"

# Watch the run in CloudWatch Logs (stream prefix "web"):
aws logs tail /ecs/finyl-dcp --follow --region "$REGION"
# Expect in the migration output: "Done: applied 15, skipped 0." on the first run.
```

(Optional) seed a fresh pilot. To seed RBAC roles + a demo tenant/admin (FRESH pilot only — never after migrating real data), run another one-off task overriding the command with `python -m app.seeds.seed` (review `app/seeds/seed.py` first).

Alternative — bastion / local psql. If you prefer, run the two commands from a host with network reachability to RDS (an EC2 bastion in the VPC, or temporarily via a publicly-accessible RDS locked to your IP), exactly as in Option 1 (f):

```
cd backend
export DATABASE_URL="postgresql://finyl:<DB_PASSWORD>@<RDS_ENDPOINT>:5432/finyl_dcp?sslmode=require"
export DB_SCHEMA=public
export JWT_SECRET="<SAME_JWT_SECRET_AS_SECRETS_MANAGER>"
AUTO_CREATE_TABLES=true ./venv/bin/python -c "
from app.core.database import Base, engine, ensure_schema
from app import models # noqa: registers all tables
ensure_schema(); Base.metadata.create_all(bind=engine)
print('base tables created')
"
./venv/bin/python scripts/run_migrations.py # Expect: "Done: applied 15, skipped 0."
cd ..
```

`run_migrations.py` applies `migrations/*.sql` in order, idempotently, tracking each in a `schema_migrations` table — re-running is safe and only applies new files.

(k) Migrate data from Abacus (only if carrying existing data)

```
# 1) Dump from the CURRENT Abacus database. The repo ships deploy/backup_db.sh which
# produces a pg_dump -Fc custom-format dump; or run pg_dump directly:
pg_dump "postgresql://<ABACUS_DB_URL>" -Fc -f finyl_dcp_cutover.dump

# 2) Restore into RDS. The Abacus DB uses schema "finyl_dcp"; RDS here uses "public".
# Remap the schema on restore so objects land in public:
pg_restore --no-owner --no-privileges \
  --schema=finyl_dcp \
  -d "postgresql://finyl:<DB_PASSWORD>@<RDS_ENDPOINT>:5432/finyl_dcp?sslmode=require" \
  finyl_dcp_cutover.dump
# If object names collide with the public schema, restore into a finyl_dcp schema on
# RDS instead (set DB_SCHEMA=finyl_dcp on BOTH services) – keep source and target
# schema names aligned. Do NOT also run section (j): the dump already contains the
# full, migrated schema + data.
```

CRITICAL — crypto keys must match. Copy `FIELD_ENCRYPTION_KEY`, `PII_ENCRYPTION_KEY`, and `JWT_SECRET` from the Abacus `backend/.env` into the corresponding `finyl-dcp/*` Secrets Manager secrets **unchanged** (step d). If they differ, encrypted PII (e.g. `borrowers.national_id`) will NOT decrypt and all existing user sessions are invalidated. After the services are up, verify decryption: log in and open a client that has a National ID — it must display in cleartext (transparently decrypted on read).

(l) Point the Netlify frontend at the ECS backend (proxy)

The recommended wiring is a **same-origin** `/api/*` **proxy** in `netlify.toml`, already added by this option's edit. The browser calls the Netlify domain; Netlify forwards `/api/*` server-side to the ALB. **No CORS to configure and the CSP `connect-src` can stay `'self'`** — the browser never talks to the ALB origin directly.

1. In Netlify: **Add new site → Import from Git** → pick the GitHub repo. The root `netlify.toml` sets base frontend, build `npm run build`, publish `frontend/dist`, the `/api/*` proxy, the SPA fallback, and security headers.
2. **Edit the proxy destination** in `netlify.toml`: replace `https://REPLACE-WITH-YOUR-ALB-OR-DOMAIN` with your API host — the custom domain you pointed at the ALB (`https://api.yourbank.co.ke`) or `https://<ALB_DNS>`. Keep the `/api` prefix and `:splat`. Commit and let Netlify rebuild.
3. Because requests are same-origin, **leave `VITE_API_URL` blank** (the frontend builds relative `/api/...` calls) and **leave the CSP `connect-src` `'self'` as-is**.
4. Deploy. Note the Netlify site URL, e.g. `https://<YOUR_SITE>.netlify.app`.

Alternative — direct cross-origin calls (no proxy). You may instead point the browser straight at the ALB: set `VITE_API_URL=https://api.yourbank.co.ke` in Netlify, add the Netlify origin to the backend `ALLOWED_ORIGINS`, **and** add the ALB/API origin to the CSP `connect-src` in `netlify.toml`. This reintroduces CORS and a browser-visible backend origin — the proxy above avoids both, so it is the recommended path. The backend parses `ALLOWED_ORIGINS` into an explicit CORS allowlist (never a wildcard with credentials).

(m) Point M-Pesa / Daraja at the new backend + go-live env flips

1. Set the platform callback base URL to your API domain (or `https://<ALB_DNS>`) via `DARAJA_CALLBACK_BASE_URL` in **both** task definitions' environment, re-register the task definitions, and update the services (`aws ecs update-service --cluster "$CLUSTER" --service-name finyl-web --task-definition finyl-web --force-new-deployment`, likewise for `finyl-scheduler`).
2. **The four Daraja webhook endpoints** (prefix `/api/v1/payments`, with the unguessable `MPESA_CALLBACK_TOKEN` path segment) are:

```
POST https://<BACKEND_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/b2c-result
POST https://<BACKEND_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/b2c-timeout
POST https://<BACKEND_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/stk-callback
POST https://<BACKEND_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/c2b-callback
```

1. **Per tenant**, update the registered Daraja callback URLs (STK, C2B validation/confirmation, B2C result/timeout) in the Safaricom portal / via the app's integration config to use the new `<BACKEND_URL>`. Tenant Daraja credentials live encrypted per-tenant in the DB — they migrate with the data (section k), so only the URLs change.
2. **At go-live**, flip these env vars (in both task definitions, then re-register + `--force-new-deployment`):
 - `DARAJA_ENVIRONMENT=production` (once you have production credentials),
 - `SAFARICOM_IP_ENFORCE=enforce` (rejects non-Safaricom source IPs on callbacks),
 - `DARAJA_CALLBACK_BASE_URL` and `ALLOWED_ORIGINS` set to the final domains.

See `deploy/DARAJA_GO_LIVE.md` for the full checklist.

(n) Autoscaling (finyl-web only)

Scale the **web** service on CPU with Application Auto Scaling target tracking:

```
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/$CLUSTER/finyl-web \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 1 --max-capacity 4 --region "$REGION"

aws application-autoscaling put-scaling-policy \
  --service-namespace ecs \
  --resource-id service/$CLUSTER/finyl-web \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-name finyl-web-cpu-tt \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "TargetValue": 60.0,
    "PredefinedMetricSpecification": {"PredefinedMetricType": "ECSServiceAverageCPUUtilization"},
    "ScaleInCooldown": 120, "ScaleOutCooldown": 60
  }' \
  --region "$REGION"
```

⚠ **NEVER autoscale finyl-scheduler.** Its `desiredCount` must stay **exactly 1**. Because the scheduler runs in-process, every extra scheduler task would duplicate every background job (double reconciliation, duplicate webhook retries). Only `finyl-web` (which has `SCHEDULER_ENABLED=false`) is safe to scale.

(o) Cost awareness (rough estimates — verify current pricing)

Unlike the Abacus deployment, these AWS resources are billed to **your** AWS account:

- **ECS Fargate tasks:** billed per vCPU-second + GB-second while running. `finyl-web` at 0.5 vCPU/1 GB and `finyl-scheduler` at 0.25 vCPU/0.5 GB, both always-on, are the compute cost. Consider `FARGATE_SPOT` for non-critical capacity.
- **Application Load Balancer:** an **always-on hourly charge + LCU usage**. This is the main **fixed** cost that App Runner did not have — the price of running in a region without App Runner. Budget for it explicitly.
- **RDS `db.t4g.micro` + 20 GB `gp3`:** on the order of ~US\$12-15/month.
- **NAT gateway** (only if you choose the private-subnet model in b.3): additional hourly + per-GB charge. The public-subnet pilot model avoids it.
- **ECR:** image storage is a few cents/GB-month.
- **Secrets Manager:** ~US\$0.40 per secret/month.
- **CloudWatch Logs / S3 backups:** pennies at pilot volumes.

All figures are estimates for planning only — confirm against the AWS pricing pages for `<REGION>`. The ALB + two always-on Fargate tasks make this option's baseline cost somewhat higher than Option 1's App Runner, which is the expected tradeoff for `af-south-1`.

(p) PRE-CUTOVER VERIFICATION CHECKLIST + decommission

Run **all** of these against the AWS deployment before sending real traffic. Keep the Abacus deployment running in parallel until every item passes.

- [] **Both services healthy.** `aws ecs describe-services --cluster "$CLUSTER" --services finyl-web finyl-scheduler --region "$REGION"` shows `runningCount` matching `desiredCount` (web ≥ 1 , scheduler exactly 1).
- [] **Target healthy.** `aws elbv2 describe-target-health --target-group-arn "$TG_ARN" --region "$REGION"` shows the web task(s) healthy.
- [] **Health.** `curl -s https://<BACKEND_URL>/api/health` → `{"status":"ok",...}` (HTTP 200).
- [] **HTTP→HTTPS.** `curl -sI http://<BACKEND_URL>/api/health` returns a 301 to https.
- [] **Login.** Authenticate via the Netlify frontend (or `POST /api/v1/auth/login`) and receive a token.
- [] **Client + PII decrypt.** Create a client with a National ID, then read it back — the `national_id` displays in cleartext (proves the crypto keys are correct).
- [] **Scheduler running (once).** The finyl-scheduler task logs show the scheduler started (`SCHEDULER_ENABLED=true`); the finyl-web logs show it **disabled**.
- [] **Daraja callback (sandbox).** Post a sandbox callback to `/api/v1/payments/mpesa/<TOKEN>/stk-callback` → HTTP 200 and a row is written to the webhook events table (`mpesa_webhook_events`).
- [] **Migrations tracked.** `SELECT filename FROM schema_migrations ORDER BY filename;` lists all applied migration files (15 for a fresh bootstrap).
- [] **Backups.** Run `deploy/backup_db.sh` (as a one-off task or from a bastion) → a dump is produced AND uploaded to the S3 bucket.
- [] **Proxy/CORS.** The Netlify frontend calls `/api/...` with no browser error (proxy path: CSP `connect-src 'self'` suffices).

Once all pass: point users at the Netlify + ECS/ALB stack, monitor for a parallel-run window, then **decommission Abacus** (stop the Abacus deployment). Keep a final Abacus DB dump archived before shutting it down.

Related docs

- `deploy/AWS_OPTION1.md` — Option 1 (App Runner) runbook, for regions where App Runner is available.
- `deploy/aws/ecs-task-web.json` — the finyl-web Fargate task definition edited in section (h).
- `deploy/aws/ecs-task-scheduler.json` — the finyl-scheduler Fargate task definition edited in section (h).
- `deploy/RUNNING_INDEPENDENTLY.md` — host-agnostic run/deploy (Docker, split deploy, backups).
- `deploy/DISASTER_RECOVERY.md` — backup/restore and recovery procedures.
- `deploy/DARAJA_GO_LIVE.md` — Safaricom Daraja production go-live checklist.